Qualité de développement

Tests unitaires en TypeScript

Mise en place

Nous allons mettre à jour votre environnement pour pouvoir utiliser les tests unitaires en TypeScript. Ceci n'est à faire qu'une fois.

1. Placer vous de votre répertoire vous servant pour vos cours de développement orienté objet et de qualité de développement.
2. Installation les paquetages :

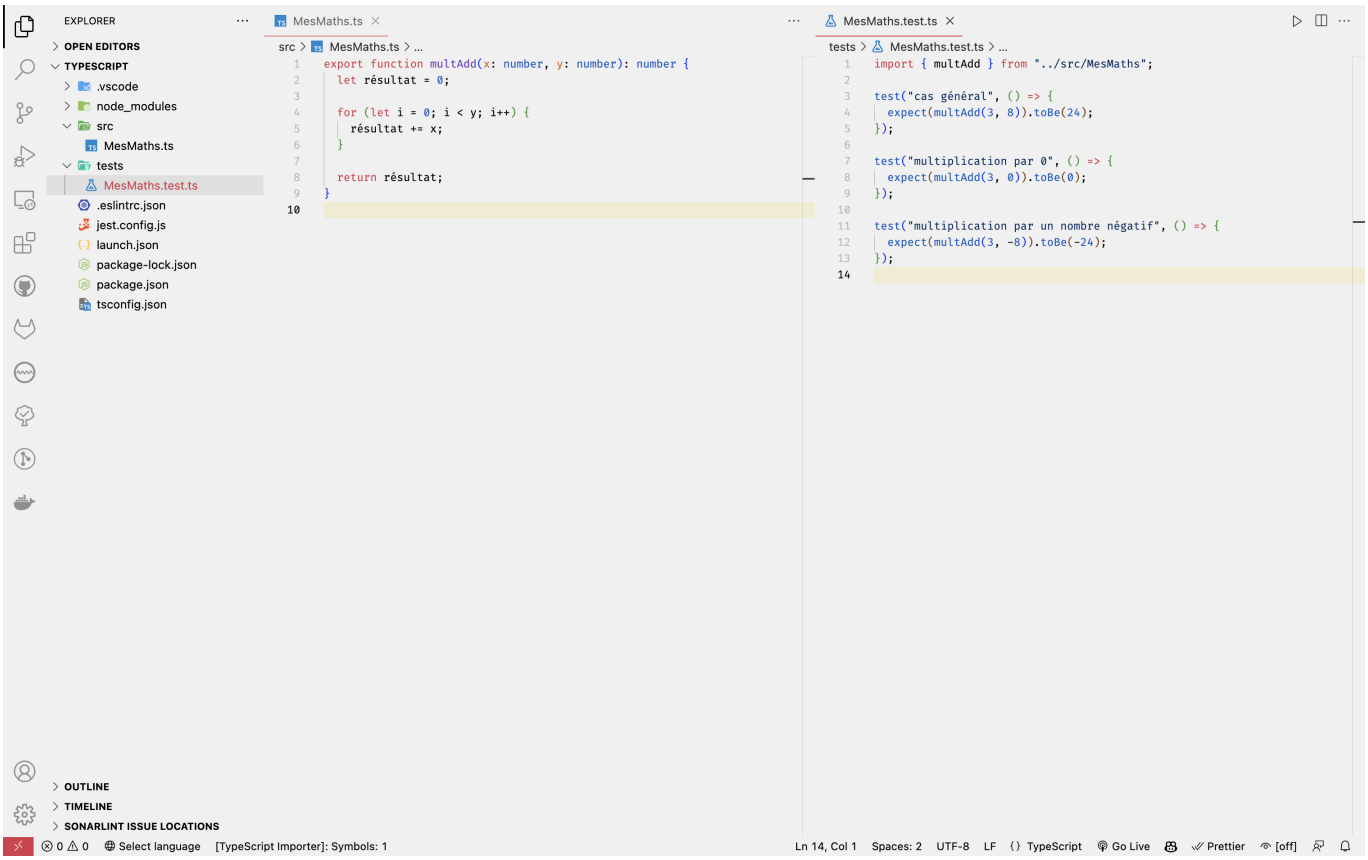
```
npm install --save-dev jest ts-jest @types/jest
```

3. Créer un fichier de configuration

```
npx ts-jest config:init
```

Bonnes pratiques

Il est conseillé de séparer les fichiers contenant le code des fichiers contenant les tests. Vous êtes encouragé à créer un répertoire `src` ou `metier` ou `app` pour y placer vos fichiers de code et un répertoire `tests` pour y placer vos fichiers de tests. Vous devriez avoir quelque chose comme ceci :



Remarque : il faut modifier le chemin vers `MesMaths.ts` dans le fichier `MesMaths.test.ts` pour qu'il soit correct par rapport au cours.

Lancer la commande `npx jest` pour vérifier que les tests fonctionnent.

Exercice 1

Corriger la fonction `multAdd` pour que l'ensemble des tests passent avec succès.

Exercice 2

Dans le fichier `MesMaths.ts`, ajouter la fonction suivante :

```
export function puissanceMult(x: number, y: number): number {
  let val = 1;

  for (let i = 0; i < y; i++) {
    val *= x;
  }

  return val;
}
```

On considère dans un premier temps que les paramètres sont nécessairement des entiers.

1. Quels sont les cas à tester ? Pour chacun donner le résultat attendu.

2. Ajouter les tests correspondants dans le fichier `puissanceMult.test.ts`.
3. Lancer les tests avec `npx jest`.
4. Lancer les tests avec `npx jest tests/puissanceMult.test.ts`.
5. Quelle est la différence ?
6. Quel cas pose problème ?
7. Modifier la fonction `puissanceMult` pour qu'elle lève une exception définie ci-dessous avec un message adéquate lorsque ce cas se présente.

```
export class ErreurPuissanceMult extends Error {
  constructor(msg: string) {
    super(msg);
    this.name = "ErreurPuissanceMult";
  }
}
```

6. Modifier votre fichier de test pour qu'il prenne en compte cette nouvelle exception. Vous trouverez une introduction à l'ensembles des possibilités de tests à l'adresse suivante : https://jestjs.io/fr/docs/using-matchers. Une liste complète se trouve à l'adresse https://jestjs.io/docs/expect.
7. Traiter le cas dans la fonction `puissanceMult` avec les tests adéquats pour vérifier que les paramètres sont bien des entiers. Vous pouvez utiliser la fonction `Number.isInteger` pour cela :

```
let x = 2;
let y = 3.14;

console.log("Number.isInteger(x) =", Number.isInteger(x));
console.log("Number.isInteger(y) =", Number.isInteger(y));
```

```
Number.isInteger(x) = true
Number.isInteger(y) = false
```

Exercice 3

Tester la fonction `creeInitiales` de l'exercice 1 du TD sur les exceptions.

Exercice 4

Tester la fonction de l'exercice 2 du TD sur les exceptions.

Exercice 5

Tester la méthode `setINE` de l'exercice 3 du TD sur les exceptions. Vous devrez notamment tester les cas où l'INE est invalide et s'il est valide que l'objet contient la bonne valeur (vous pouvez ajouter un getter pour cela).